

Provenance

Product Traceability with a Simulated Hardware Authenticator

Technical report

Hachem Nasri

Independent project · Kasserine, Tunisia

phigor.arc@gmail.com

Abstract

A blockchain can record a product's history, but it cannot directly observe whether a physical item is genuine. Provenance explores this boundary through a working Ethereum prototype that combines role-based product workflows, a modular contract router, and challenge–response authentication. Brands enroll an authenticator address for each product. A verifier accepts a signature only when it matches that address and the product's challenge has not already been consumed. The implementation uses a software key service to simulate the authenticator; it does not demonstrate a physical unclonable function (PUF).

This technical report asks what contract-verified authentication establishes and where trust in the physical product enters the system. A valid response authenticates the registered signing key; interpreting it as evidence of product authenticity additionally assumes correct enrollment and a protected authenticator that remains connected to the genuine item. The report describes the implemented system, these trust assumptions, and executable tests for both intended behavior and security limits. It distinguishes replay rejection within one deployment from live relay, first-submitter races, and reuse across deployments. The contract architecture follows the Diamond routing pattern, but full ERC-2535 and ERC-721 conformance is not claimed. The contribution is an inspectable systems prototype and an evidence-based analysis of its boundaries, rather than a new cryptographic primitive or a production-ready anti-counterfeiting system. The results support further work on hardware enrollment, domain-separated signatures, and consistent lifecycle authorization.

Keywords: product provenance; Ethereum; smart contracts; PUF simulation; replay protection; role-based access control.

Contents

1	Introduction	2
2	Background and related work	2
3	System model and trust assumptions	3
4	Implementation architecture	4
5	Product lifecycle	6
6	Authentication protocol	7
7	Evaluation	8
8	Discussion and future work	10
9	Conclusion	11
	References	12

1 Introduction

A product identifier answers “which record should I look up?” It does not, by itself, answer “is this the physical product described by that record?” A copied label can point to a genuine record. A ledger can preserve the submitted history while the item in front of the buyer is counterfeit. Provenance starts from this distinction. The project combines two mechanisms. First, authorized participants record manufacturing, shipment, receipt, return, and sale operations. Second, an authenticator signs a challenge associated with a product identifier. The contract checks this signature against the address registered by the brand. Together, these mechanisms make recorded custody and key-based authentication inspectable through one contract address.

A PUF is a possible foundation for the authenticator: hardware variation can help reconstruct a device-specific secret. However, implementing a signature check is not equivalent to implementing a PUF. The present project uses ordinary software keys to evaluate the contract interface and workflow. The distinction matters because software testing cannot establish resistance to key extraction, hardware cloning, or physical tag replacement.

1.1 Research questions and contribution

The central research question is: **What does a contract-verified authenticator response establish about a product, and which physical-authenticity claims still depend on enrollment and hardware trust?** The custody workflow provides the experimental setting. Three supporting questions organize the investigation:

1. **RQ1 — Workflow:** How can one contract system organize role-controlled product transitions and a shared product history?
2. **RQ2 — Authentication:** What exactly does the implemented signature-and-nonce check establish, and which assumptions are needed to connect an authenticated key to the physical item being presented?
3. **RQ3 — Evidence:** Which properties can be exercised by local contract tests without claiming hardware validation or complete security?

The contribution is a concrete implementation and analysis of these questions. It includes a map from the protocol description to source files, an explicit lifecycle, an exact description of the signed bytes, and executable boundary tests. It does not claim that combining blockchain records, digital twins, and PUF-backed identities is new. Its value is making the integration and its limitations easy to inspect and reproduce.

1.2 Why use a blockchain here?

A public ledger lets different organizations read the same accepted records and inspect which address submitted a transaction. Contract rules can reject some unauthorized updates before they become accepted state. These properties are useful when participants want a shared history with externally visible rules.

There are costs. Transactions require fees and confirmation time, addresses and product activity are public, and the upgrade owner remains trusted. A conventional database with authenticated access and signed audit logs may be simpler when one organization controls the whole process. This report does not claim that a blockchain is necessary for every supply chain or that centralized databases cannot support audits.

2 Background and related work

2.1 Supply-chain records and physical identity

Casella et al. review applications of blockchain in supply chains [1], while Agarwal et al. review blockchain-based supply-chain security [2]. These works provide context for the project: a shared ledger is one part of a wider system involving organizational processes, data collection, and access control. Provenance studies a small, concrete workflow within that setting.

Dietrich et al. examine a traceability architecture for object-related supply-chain events [3]. This is relevant to recording custody and lifecycle changes. Provenance additionally connects each product record to a registered signing address, while keeping the distinction between recorded events and physical evidence explicit.

2.2 Hardware authentication

Herder et al. explain PUF-based authentication and key generation, including the need to account for noisy hardware responses [4]. Their tutorial helps explain why a real device requires more than a software-generated key. The present simulation implements the signing interface, not the physical key-reconstruction process.

Arcenegui et al. study binding IoT devices to smart non-fungible tokens through PUF-derived identities and demonstrate their approach with ESP32-based hardware [5]. That work is directly relevant to the intended hardware boundary. Provenance instead evaluates software signing, product custody operations, and a modular contract layout. It therefore cannot inherit the hardware evidence of that study.

Negka et al. investigate counterfeit IoT device detection by combining PUF-based component identification with blockchain tracking [6]. Their work reinforces the need to distinguish the identity of a hardware component from the history recorded about it. Provenance concentrates on the contract-side workflow and uses a simulated authenticator rather than evaluating physical components.

2.3 Modular contracts and upgrades

The Diamond standard describes routing calls from a stable address to multiple implementation contracts, called facets [7]. Provenance adopts this routing pattern to separate responsibilities. This is an engineering choice, not a throughput improvement. EIP-170 limits the deployed runtime code of an individual contract to 24,576 bytes [8]. Splitting logic into facets can distribute code across contracts, but each deployed facet still has its own size constraint.

Upgradeability also adds trust and implementation risks. Bodell et al. examine proxy-based upgradeable contracts and their security implications [9]. In Provenance, the owner can replace routed code. The report therefore treats upgrade control as part of the trust model rather than describing the application as fully trustless.

Reference / approach	Relevant idea	Boundary of this project
PUF-bound smart NFTs	Device identity backed by hardware reconstruction of a secret.	No physical PUF, reliability study, or extraction-resistance experiment.
Diamond routing	Separate implementations behind one address and shared state.	Routing is implemented; complete standard conformance is not established.
Proxy security studies	Upgrade authority and implementation safety require analysis.	Local tests cover selected behaviors, not all possible upgrades.

The comparison is deliberately narrow. No comparative gas, throughput, or hardware-security experiment is presented, so none of these approaches is ranked as faster or more secure.

3 System model and trust assumptions

3.1 Participants and assets

An administrator registers brands and controls their status. An active brand mints product records and registers retailers. A retailer receives, returns, or sells products through its permitted functions. Consumers read records and can submit authenticator signatures. A separate diamond owner controls upgrades; this authority is distinct from an application role.

Each product has a token identifier, a name, a brand address, an authenticator address, mint and sale timestamps, and a lifecycle status. Ownership and balances are recorded separately. The authenticator field stores an Ethereum **address**, not a full elliptic-curve public key. Address recovery from a signature allows the contract to compare the signer with this stored value.

The important assets are the integrity of product records, authorization to change custody, control of authenticator secrets, and integrity of the routing and storage layout. Physical goods themselves remain outside the EVM.

3.2 Adversary and assumptions

An external attacker may read public chain data, copy labels, create transactions, submit arbitrary signatures, and observe a valid signature before it is mined. A dishonest participant may also try operations outside its assigned role. The analysis assumes normal EVM execution and the expected security of the signature and hash primitives.

For a physical deployment, enrollment must correctly bind a genuine product to an authenticator. The signing secret must remain protected, and the device must remain attached to the intended product. Those are assumptions of the proposed hardware setting, not properties demonstrated by this software implementation. This is the boundary between **cryptographic authentication** and **physical authenticity**. The contract checks a response against an enrolled address; it cannot inspect how the authenticator is attached or whether the item presented is the enrolled product. Moving a genuine authenticator to a counterfeit item, or relaying a fresh response from a genuine device elsewhere, can leave the signature check valid while breaking that physical interpretation. A successful check therefore does not independently establish product authenticity or physical proximity.

An attacker who obtains the registered secret can generate signatures that the verifier accepts. A malicious authorized brand can register an attacker-controlled address. An upgrade owner can change the validation code. None of these threats is solved by recording a signature on-chain.

3.3 Demonstration boundary

The current backend stores software private keys in MongoDB and returns them through a key-retrieval endpoint. The browser uses those keys to simulate device signing. The interface can display the private key for demonstration. These choices make the workflow visible, but they do not protect device secrets.

In particular, the backend has no application authentication on its key endpoints. CORS restrictions are not an authorization mechanism against a direct HTTP client. Anyone able to retrieve a simulated product key can act as that product's authenticator. This service must not hold real production device secrets.

The separate demo faucet lets visitors request roles when it has the required permissions. It intentionally relaxes admission control. A demonstration with that faucet enabled cannot also claim that only independently vetted organizations can become administrators.

Interpretation of “verified”. In this prototype, a successful transaction means that the contract accepted a signature for a registered address and an unused product challenge. It does not mean that a physical product was independently inspected or that a device was physically near the buyer.

4 Implementation architecture

4.1 Router, facets, and shared storage

The public entry point is `smart-contracts/contracts/Provenance.sol`. A call's first four bytes select a function. The fallback function looks up that selector and executes the corresponding facet using `delegatecall`. Facet code then reads and writes the router's storage, preserving the original transaction sender.

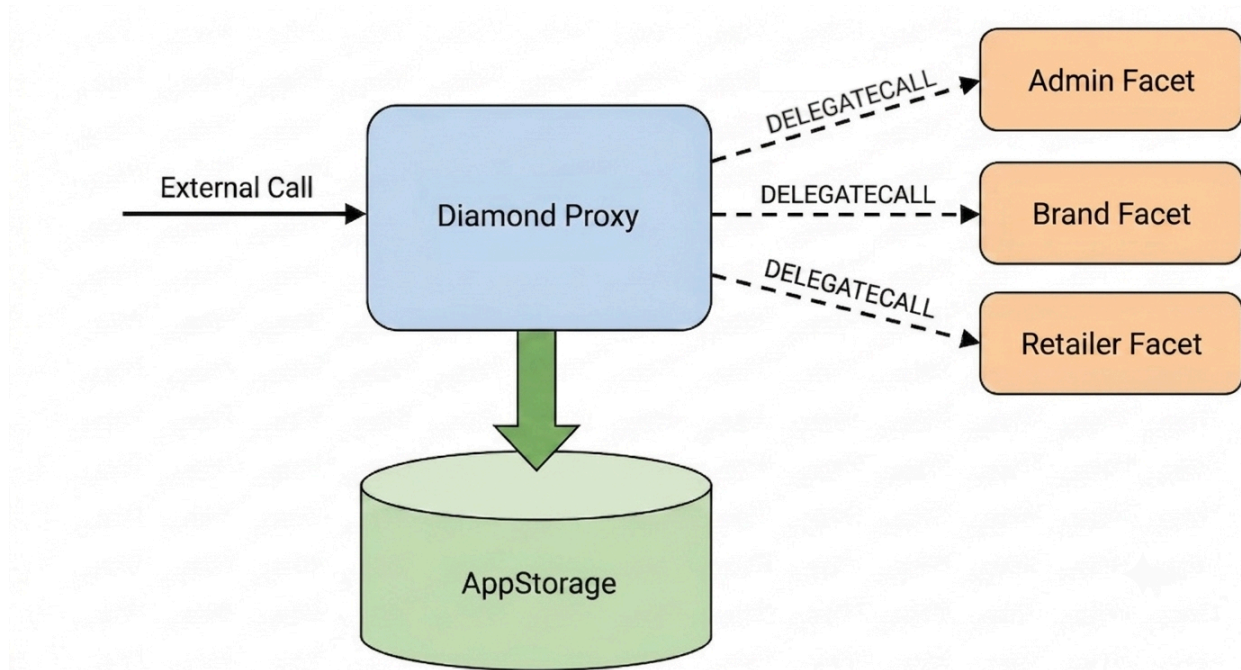


Figure 1: Diamond routing and shared application storage. Facets execute through the proxy; storage compatibility must still be preserved during upgrades.

`LibAppStorage.appStorage()` returns a storage reference anchored at the hash of "diamond.standard.app.storage". `LibDiamond` uses a different hashed slot for routing state. A Solidity storage reference points to persistent contract state; it is not a new copy of the struct in temporary memory. Shared storage makes it possible for the brand and retailer functions to update the same product. It also creates coupling: an incompatible layout change can corrupt the interpretation of existing data. Distinct root slots do not automatically make future upgrades safe. Storage compatibility and upgrade authorization require separate review.

The six facets separate brand administration, brand operations, retailer operations, consumer verification, role management, and ownership queries. Shared libraries implement routing, role checks, and ownership changes. This division organizes the code without separating each actor's product records into a different ledger.

Despite the name `ERC721Facet`, its public functions are limited to four queries. It does not expose the full standard transfer, approval, and interface-detection API. This report describes an NFT-like ownership record rather than asserting full ERC-721 compatibility [10].

Likewise, the router implements the Diamond pattern but lacks the standard loupe inspection functions in the inspected facet set. `LibDiamond` also does not establish all the validation guarantees expected of a hardened implementation. Full ERC-2535 compliance should not be inferred from the use of `diamondCut`.

4.2 Roles and administrative control

Application roles are `DEFAULT_ADMIN_ROLE`, `ADMIN_ROLE`, `BRAND_ROLE`, and `RETAILER_ROLE`. Membership is one condition; brand and retailer profile status can be another. For example, possessing the brand role does not make a pending brand active.

The constructor assigns default-administrator and administrator roles to the deployer, while its owner argument sets the diamond upgrade owner. These can be different addresses. Public `grantRole` first requires default-administrator membership and then calls the library, which checks the role's configured administrator. Thus, it is not enough to describe public grants as a simple unrestricted role hierarchy.

The storage slot and access checks are small and readable, but readability is not proof of complete policy enforcement. Return operations and cross-brand shipment behavior deserve particular attention, discussed below.

5 Product lifecycle

The lifecycle is a directed graph with an intentional return path. It is not a strictly forward-only sequence.

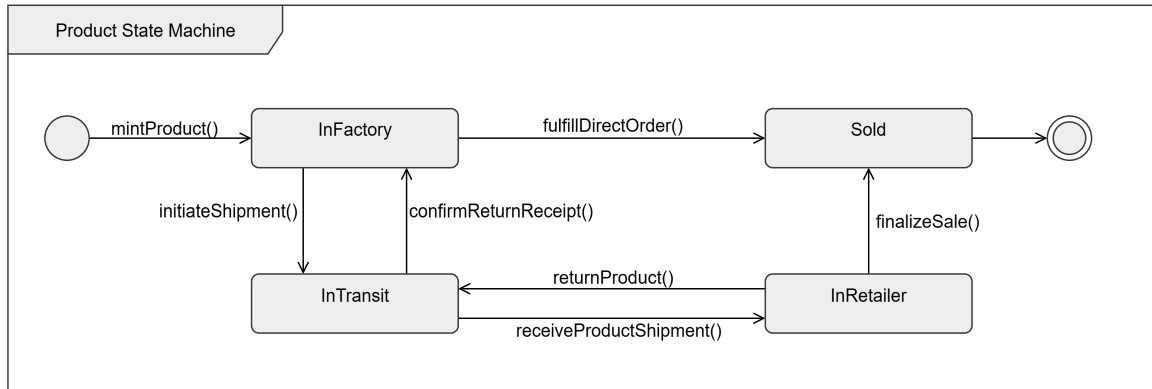


Figure 2: Product lifecycle with direct sales and a return route. The diagram’s returnProduct label corresponds to the implemented batch function, returnProducts.

Before	Operation	After	Ownership effect
No record	mintProduct	InFactory	Assigned to brand
InFactory	initiateShipment	InTransit	Brand → retailer
InTransit	receiveProductShipment	InRetailer	Remains retailer
InRetailer	finalizeSale	Sold	Retailer → consumer
InFactory	fulfillDirectOrder	Sold	Brand → consumer
InRetailer	returnProducts	InTransit	Retailer → parent brand
InTransit	confirmReturnReceipt	InFactory	Remains brand

The same transit state represents outbound shipment and a return. Ownership and caller checks help distinguish who may receive it. Ownership changes when shipment is initiated, before the receiving party confirms physical receipt. Therefore, on-chain ownership is a workflow record, not a sensor measurement of current physical possession.

Sold has no outgoing transition in the inspected public product workflow. This is a property of the current function set, not an irreversible constraint on future owner-authorized upgrades.

5.1 Transaction atomicity

Shipment and receipt functions operate on arrays of identifiers. If a later item fails a check, the EVM reverts earlier writes in that same transaction. For example, sending the duplicate batch [1, 1] changes the first item’s status during the first loop iteration, then fails on the second. The test checks that ownership and status both remain at their pre-call values.

This establishes an observable all-or-nothing result for the tested batch. It does not mean that arbitrary batch sizes are practical: gas consumption grows with the number of items, and sufficiently large transactions cannot fit within network limits.

5.2 Policy inconsistencies in the current code

The function comments describe active participants for returns, but returnProducts checks the retailer role without checking retailer status or parent-brand status. confirmReturnReceipt checks the brand role without an active-brand status check. A suspended participant that retains the relevant role can therefore retain return-

related capabilities. This might be a deliberate recovery policy, but the current description does not specify it as one.

`initiateShipment` checks that the recipient retailer exists and is active, but does not check that the retailer's parent brand is the sending brand. Since `returnProducts` sends goods to the retailer's recorded parent brand, cross-brand shipment needs an explicit policy before a deployment can be considered safe for real inventory. Characterization tests exercise both this ownership outcome and the suspended-participant return path against the unchanged contracts.

6 Authentication protocol

6.1 Enrollment and message construction

During minting, the brand passes a nonzero authenticator address. The contract does not prove how that address was generated or enforce that each product uses a different key. Reusing a key across products is possible; the token identifier in the signed message separates their messages.

Let t be the product identifier and c the challenge, both represented as `uint256`. The implemented helper is:

```
function getVerificationHash(
    uint256 tokenId,
    uint256 challenge
) internal pure returns (bytes32) {
    return keccak256(abi.encodePacked(tokenId, challenge));
}
```

Each input occupies 32 bytes, so the packed message contains 64 bytes. This fixed-width encoding does not have the ambiguity associated with concatenating several variable-length strings.

The application signs the 32 hash bytes, rather than the characters of a hexadecimal string:

```
const hash = ethers.solidityPackedKeccak256(
    ["uint256", "uint256"], [tokenId, challenge]
);
const signature = await authenticator.signMessage(
    ethers.getBytes(hash)
);
```

`signMessage` applies the Ethereum signed-message prefix. The contract's `toEthSignedMessageHash()` applies the corresponding transformation before recovery, following the personal-message form of ERC-191 [11]. If the client signs the text `"0x..."` instead of the underlying bytes, the recovered signer will not match the intended message.

6.2 Verification and nonce consumption

Verification runs through `ConsumerFacet.consumeVerification`. The following steps execute in one state-changing transaction:

1. Checks whether `isNonceUsed[tokenId][challenge]` is already true.
2. Computes the hash described above.
3. Recovers the signer using OpenZeppelin's ECDSA helper.
4. Compares the recovered address with the product's enrolled authenticator.
5. Marks this product–challenge pair as used.
6. Emits `ProductVerified(tokenId, challenge)`.

An invalid signature reverts before the nonce is consumed. Malformed signatures may revert inside the cryptographic helper rather than with the application's signer-mismatch error. The used-nonce write and event happen only if all preceding checks succeed.

Unlike an off-chain read or `eth_call` simulation, a successful mined transaction persists the used-nonce state. A client-side pre-check is useful feedback, but it does not reserve the nonce: another transaction may consume it first.

6.3 Precisely scoped replay property

Under the current code and unchanged nonce state, after a successful verification of product `t` with challenge `c`, a later call with that same pair reverts. The first successful call stores `true`; the next begins by checking that stored value. This is a local state argument, not a claim that every replay or impersonation attack is impossible. The nonce mapping is per product. Two products may use the same numeric challenge, but each requires its own correctly bound signature. Conversely, changing the product identifier or challenge while retaining the original signature fails in the tested cases.

6.4 Remaining attacks and interpretation

Live relay. An attacker can forward a fresh challenge to a real signer and return its response. The contract has no distance measurement or authenticated physical channel. Nonce consumption does not establish physical proximity.

First-submitter race. The message does not contain the submitting account. An observer can submit a copied valid signature before its intended sender. The first transaction succeeds and consumes the nonce; the second reverts. This does not change the registered key, but it can disrupt the intended verification session.

Cross-deployment reuse. The message contains neither chain ID nor contract address. Another deployment with the same token identifier and enrolled authenticator can accept the same signature because its nonce state is independent. A test demonstrates this with two local deployments. Cross-chain reuse follows from the same missing binding, but is not separately measured on two networks.

Delayed first use. There is no signed expiry time or on-chain challenge issuance. An otherwise valid unused signature remains acceptable later. “Not previously consumed” is therefore weaker than “recently generated.”

Challenge generation. The current browser uses `Math.random()` over one billion values. The contract accepts the full `uint256` range, including zero, but the UI does not use that range or a cryptographically secure generator. Randomness does not replace domain binding, expiration, or nonce state.

A future message format should bind the deployment, chain, action, product, challenge, and any intended verifier or expiry policy. That would be a protocol change requiring matching changes to the signer and tests. It is not presented here as already implemented.

7 Evaluation

7.1 Method

The evaluation uses the actual Solidity contracts and a local Hardhat EVM. The test fixture deploys six facets, builds their selector list, deploys the router, and attaches each facet ABI to the router address. Tests therefore exercise delegated calls against shared router storage rather than calling isolated facets as if each were a complete application.

Authenticators in the tests are software signers. No PUF device, network adversary testbed, or production workload is involved. Each test receives a fresh deployment through the fixture. This avoids accidental dependence on the nonce state or role changes of an earlier case.

The environment used for this revision is Node.js 24.12.0, Hardhat 3.3.0, Solidity 0.8.28, and Mocha 11.8.0 on Windows. The dependency versions are recorded in `smart-contracts/package-lock.json`. The default compiler profile is used. The production profile separately enables the optimizer with 200 runs; results from the default profile should not be silently presented as optimized deployment measurements.

Akca et al. evaluate smart-contract testing techniques using both coverage and fault-finding ability [12]. These are different measures: executing a statement does not establish that an assertion would detect an incorrect result. The evaluation below therefore reports coverage alongside behavioral checks and tests that reproduce known limitations.

7.2 Test groups and what they establish

Group	Examples of checks	Interpretation
Setup and roles	Initial roles; unauthorized operations; demo role requests.	Selected authorization and setup behavior.
Product lifecycle	Mint, shipment, receipt, returns, direct and retail sale.	Expected state and ownership changes for fixture scenarios.
Verification	Correct signer, wrong signer, used nonce, missing product.	Selected acceptance and rejection paths.
Message boundaries	Changed challenge, changed token, malformed signature, zero and maximum challenge.	Encoding and state-boundary behavior.
Nonce isolation	Same number on two products; rejected input leaves nonce unused.	Per-product state and rollback behavior.
Characterization	Copied signatures; another deployment; suspended returns; cross-brand returns.	Known limits are reproduced, not “fixed” by passing tests.
Batch and upgrade	Duplicate batch rollback; non-owner upgrade rejected.	Atomic rollback and one upgrade authorization boundary.

The new boundary cases deliberately include tests that **succeed when an undesirable capability exists**. For example, accepting a signature in a second deployment documents missing domain separation. Such a passing test is evidence of a limitation, not evidence that the attack has been prevented.

7.3 Results and limits of the evidence

The local evaluation completed with **81 passing test cases and no failures**: 69 existing cases and 12 added boundary and characterization cases. TypeScript checking with `npx tsc --noEmit` also completed successfully. These are test cases, not 81 independent test suites.

The same 81 cases also passed with Hardhat’s built-in coverage enabled, using `npx hardhat test --coverage`. Table 1 reports the resulting line and statement coverage for every project source file listed by the tool. Its scope is the 13 reported files, including shared libraries and the demonstration faucet; dependency files and declaration-only files are not included in this table. The total is the tool’s aggregate over the measured source, not an arithmetic average of the displayed percentages.

Source file	Lines	Statements
Provenance.sol	93.10%	89.47%
AccessControlFacet.sol	66.67%	57.14%
AdminFacet.sol	96.55%	93.10%
BrandFacet.sol	98.98%	98.20%
ConsumerFacet.sol	94.44%	90.91%
ERC721Facet.sol	25.00%	25.00%
RetailerFacet.sol	97.87%	96.72%
LibAccessControl.sol	72.73%	72.00%
LibAppStorage.sol	100.00%	100.00%
LibDiamond.sol	61.29%	55.26%
LibERC721.sol	63.93%	56.58%
LibProvenance.sol	100.00%	100.00%
DemoRoleFaucet.sol	85.71%	76.47%
Total reported source	85.68%	81.62%

Table 1: Measured coverage for 81 passing cases, using Hardhat 3.3.0 and Solidity 0.8.28 with the default build profile. Percentages are rounded to two decimal places.

Line coverage measures which executable source lines were reached; statement coverage distinguishes executable statements, including multiple statements on one line. Neither metric shows that every possible decision outcome or input has been tested. This version of the coverage tool does not provide true branch coverage, so no branch percentage is claimed.

The lifecycle facets have high measured coverage, while ownership helpers and upgrade logic have larger gaps. For example, the suite does not exercise every selector replacement/removal path or every ownership helper. These are meaningful areas for further tests, not unimportant code that can be excluded from the total. Some uncovered defensive checks also depend on earlier guards, so an uncovered line alone does not establish that a vulnerability exists.

The tests reproduce four limitations: first-submitter signature reuse, cross-deployment reuse, return operations by suspended participants, and return ownership changing to a different parent brand. Their presence despite substantial coverage illustrates why coverage and security claims must be evaluated separately.

A passing suite means that the assertions executed successfully in the reported environment. It does not establish exhaustive branch coverage, formal correctness, hardware security, full standards compliance, or absence of vulnerabilities. The source-level policy findings in Section 5 remain relevant even when all current tests pass.

There is no controlled monolithic baseline, no measured deployment-cost comparison, and no throughput experiment. Consequently, this report makes no empirical claim that the facet architecture reduces gas or improves scalability. Those questions require separately controlled measurements.

8 Discussion and future work

8.1 What the prototype establishes

The prototype makes the relation between product workflows, role checks, and signer verification concrete. Its strongest result is an inspectable chain of reasoning: the product record supplies an authenticator address, the signed bytes bind a token and challenge, successful verification consumes per-product state, and tests demonstrate both acceptance rules and limits.

The system also shows why physical provenance is an integration problem. A correct signature check does not fix dishonest enrollment, exposed secrets, physical relocation of a genuine tag, or compromised governance. These concerns belong in the protocol’s trust model, not only in a future implementation checklist.

8.2 Prioritized next steps

First, specify and test policy. Decide whether suspended participants may complete returns, whether shipments across brand boundaries are allowed, and which authority may grant each role. Tests should express the chosen policy before any contract changes are made.

Second, bind verification to its intended context. Introduce a versioned, domain-separated message with an explicit freshness policy. Specify whether verification is intentionally open to any submitter or tied to a particular session. Preserve negative tests for modified fields and add a second-chain test.

Third, implement hardware enrollment. Select an actual PUF design or other protected authenticator, specify how a stable signing key is reconstructed, and measure reliability across environmental conditions. Evaluate physical attachment and tag transplantation separately from signature correctness. A software key service cannot substitute for these experiments.

Fourth, harden modularity. Review facet-address checks, standard inspection interfaces, selector updates, and storage compatibility. Test upgrades with populated state, not only empty fixtures. Define governance procedures that match the trust claims.

Fifth, measure costs and usability. Compare an equivalent monolithic implementation and the facet version using identical compiler settings and input sizes. Report transaction gas, runtime bytecode size, and batch-size limits. Hardware evaluation should separately report signing latency and reconstruction failures. A user study would be needed to support usability claims.

These steps are proposed work, not completed results.

9 Conclusion

Provenance is a working contract prototype for shared product records and key-based authentication. Its modular layout separates actor responsibilities, while its verifier binds signatures to product identifiers and consumes challenges within each deployment. The evidence supports these specific behaviors.

The same inspection reveals limits that matter for a real deployment: the PUF is simulated, secrets are retrievable from the demonstration service, verification lacks deployment and session binding, and some lifecycle policies are inconsistent. Stating these limits makes the contribution more precise. The project provides a reproducible starting point for studying how hardware identity, organizational trust, and blockchain state can fit together; it does not yet establish end-to-end physical authenticity.

References

- [1] G. Casella, B. Bigliardi, S. Filippelli, and E. Bottani, “Cases of application of blockchain on the supply chain: a literature review,” *Procedia Computer Science*, vol. 217, pp. 1416–1426, 2023, doi: 10.1016/j.procs.2022.12.340.
- [2] U. Agarwal *et al.*, “Blockchain Technology for Secure Supply Chain Management: A Comprehensive Review,” *IEEE Access*, vol. 10, pp. 85493–85517, 2022, doi: 10.1109/ACCESS.2022.3194319.
- [3] F. Dietrich, L. Louw, and D. Palm, “Blockchain-Based Traceability Architecture for Mapping Object-Related Supply Chain Events,” *Sensors*, vol. 23, no. 3, p. 1410, 2023, doi: 10.3390/s23031410.
- [4] C. Herder, M.-D. Yu, F. Koushanfar, and S. Devadas, “Physical Unclonable Functions and Applications: A Tutorial,” *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1126–1141, 2014, doi: 10.1109/JPROC.2014.2320516.
- [5] J. Arcenegui, R. Arjona, R. Román, and I. Baturone, “Secure Combination of IoT and Blockchain by Physically Binding IoT Devices to Smart Non-Fungible Tokens Using PUFs,” *Sensors*, vol. 21, no. 9, p. 3119, 2021, doi: 10.3390/s21093119.
- [6] L. Negka, G. Gketsios, N. A. Anagnostopoulos, G. Spathoulas, A. Kakarountas, and S. Katzenbeisser, “Employing Blockchain and Physical Unclonable Functions for Counterfeit IoT Devices Detection,” in *Proceedings of the International Conference on Omni-Layer Intelligent Systems*, ACM, 2019, pp. 172–178. doi: 10.1145/3312614.3312650.
- [7] N. Mudge, “ERC-2535: Diamonds, Multi-Facet Proxy.” [Online]. Available: <https://eips.ethereum.org/EIPS/eip-2535>
- [8] V. Buterin, “EIP-170: Contract code size limit.” [Online]. Available: <https://eips.ethereum.org/EIPS/eip-170>
- [9] W. E. Bodell III, S. Meisami, and Y. Duan, “Proxy Hunting: Understanding and Characterizing Proxy-based Upgradeable Smart Contracts in Blockchains,” in *32nd USENIX Security Symposium (USENIX Security 23)*, USENIX Association, 2023, pp. 1829–1846. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/bodell>
- [10] W. Entriken, D. Shirley, J. Evans, and N. Sachs, “ERC-721: Non-Fungible Token Standard.” [Online]. Available: <https://eips.ethereum.org/EIPS/eip-721>
- [11] M. Holst Swende and N. Johnson, “ERC-191: Signed Data Standard.” [Online]. Available: <https://eips.ethereum.org/EIPS/eip-191>
- [12] S. Akca, C. Peng, and A. Rajan, “Testing Smart Contracts,” in *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ACM, 2021, pp. 1–11. doi: 10.1145/3475716.3475779.